

CPE 470 Project Checkpoint

Justin Trinh, Kevin Fregoso

December 2025

System Overview

We attempted to create modular code for our robot, the system works by having single nodes handle sub-tasks, each node should publish needed information for future nodes and subscribe from previous nodes to get needed system information.

The file 'robot_manager' handles coordination between the nodes, publishing and subscribing in our intended system order. Our manager file intends to sequence the publishing and subscriptions of our nodes in order to coordinate the system. Through our manager file we will sequentially run our nodes, publishing the result which is later used by subscribing nodes. Thereby going sub-task by sub-task through each problem.

Each section of the report will overview the intended solution for our problems and code implementation for each subproblem.

Problem 1 Point P Navigation

Problem 1 Overview:

Our first sub-task is to localize our robot within the environment. Our environment contains three fixed landmarks which are ARUCO Markers ten, twenty, and thirty. Our robot rotates counterclockwise and uses the USB camera to detect our three known markers. Each time a marker is detected the unique id for the marker is provided as well as the cameras location from the marker. This will provide three distances from three known landmarks. The intersection of our distances provides a localized location. We then calculate the difference from our localized location to our designated

point P. The robot will then travel the distance to point P using the robots wheel encoders. This problem is split into the following sub-tasks: ARUCO detection, localization, navigation, and movement.

Problem 1 Implementation:

Our robot is to be localized within the environment in the frame of reference from point A. To localize ourself in this frame of reference we created a file title 'marker_map' this file contains the distances of our three markers from point A

Table 1: Aruco Marker Locations

Aruco Marker ID	x (m)	y (m)
30	0.000	0.465
20	0.575	0.000
10	1.150	0.465

The file titled 'aruco_detector' handles the subtask of rotating the robot counterclockwise and scanning for our three landmarks. This node publishes the distance from each of our three ARUCO markers.

The next node 'localizer' subscribes to the above node, within this node we find the intersection of our three distances, the intersection of our distances provides the localized location of our robot in the environment. We do this by defining three circles, the location of the marker is our center and the radius is our distance from the marker.

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2$$

We solve for the intersection and have that be defined as our localized location for the robot

'navigator' will subscribe to 'localizer' as well as our 'user_interface' the user interface simply asks for the x and y coordinates for point p. With the location of point p from our user interface and the localized location from 'localizer' the node will attempt to find the difference in the x and y coordinates of our two points. The straight line to point p will be the hypotenuse of the computed difference lengths.

The final node for this sub-problem takes in the computed distance from 'navigator', our node 'move_robot' will use the wheel encoders to travel the computed difference.

With the above nodes we do the following: rotate the robot to find ARUCO markers, localize ourselves using the computed distances from the markers, compute the distance from the point, move our robot to point p.

Problem 2 Exiting Environment through the gap

Problem 2 Overview

Our next sub-task is to navigate our robot out of room ABCD through the opening EF. The robot first performs a Lidar scan of the environment. The raw Lidar points are processed to find inflection points along the walls, which represent the corners of the environment walls. From these points, the program identifies the gap EF. A visibility graph is then constructed using the corners and the robot's current location, to define the free and obstacle space we use a Minkowski sum, the line segments we find and the radius of our robot is made the obstacle space and everything else as our free space which the robot can traverse through.

The shortest path from the robot's current position to a point outside the gap is computed using Dijkstra's algorithm on the visibility graph. The robot then rotates and moves along this path, following each segment in order.

This problem is split into the following sub-tasks: LiDAR scanning, processing the scan to find inflection points, line segments and our opening, path planning using a visibility graph, and movement along the computed path.

Problem 2 Implementation:

The file titled 'lidar' handles the sub-task of scanning the environment using a Lidar sensor. This node uses the Lidar to get distance measurements in polar coordinates, while filtering by discarding points one and a half meters from the Lidar since those measurements must be from outside the walled environment.

The next node, 'process_lidar' subscribes and converts from polar to cartesian then finds inflection points. The largest distance between points is our gap EF.

The 'process_lidar' file will publish its cartesian points and will be used by the 'line_segment' file to create our walls. This file creates our obstacle

environment. The line segments which represent our wall and the robots radius is our Minkowski sum, using this we have all other space to be free space and the wall segments plus the robots radius to be our obstacle space which our robot cannot move through.

The file 'visibility_graph' has our localized location, inflection points and gap as nodes for our visibility graph. This file will connect our nodes accounting for the Minkowski sum we found previously. Using Dijkstra's algorithm we move through the nodes from the localized point to our end point.

creates a graph where nodes are the start, goal, and corners of Minkowski-expanded walls. Edges connect nodes that are mutually visible, i.e., no wall intersects the line connecting them. Dijkstra's algorithm finds the shortest path from the robot's current position to the target point outside the room.

The 'path' file is responsible for implementing the movement by creating segments for each node we traverse doing:

$$\text{distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\text{angle} = \arctan \frac{y_2 - y_1}{x_2 - x_1} - \text{current orientation}$$

The segments are used in our next node 'Move_robot_follow_path' we simply rotate and go the distance we found in the previous node like we did in problem one.

With the above nodes we go path to path until we reach our final location from the visibility graph. This coordination is handled by the 'move_robot_outside_box' file which is responsible for how we sequence the publishing and subscribing of the above nodes.

Together with all the nodes, the robot performs the following sequence: scan the room with Lidar, process the point cloud to identify walls, inflection points, and the largest opening, plan a collision-free path using the visibility graph, and traverse the path to safely exit the room.

Problem 3 Target Tag

Problem 3 Overview:

This final problem occurs when the first two problems are accomplished, in this section we need to be scan for an ARUCO marker outside the enclosed environment then have the robot hit the cardboard target with the taped

marker. For this task we rotate until we are aligned with ARUCO marker 1 then head towards the marker.

Problem 3 Implementation

This relies upon problems one and two to successfully complete. We must first find and head to point p in problem one then get through the gap in problem two, this is still being managed by the 'robot_manager' file. When the manager gets the successful publication from the problem two nodes we know we have successfully exited through the gap.

When this occurs we call 'aruco_pose_parameters' this rotates the robot until it find and orientates itself with ARUCO marker 1. We then call 'move_robot_to_aruco' which will head the distance from the marker to the marker to collide with our target.

Sensors Used

Sensors used: camera, lidar sensor, odometry - wheel encoders

Individual Report

Individual Learning Experience - Justin Trinh

When I entered this class 4 months ago, I had no experience working with hardware, no knowledge about robotics, and had never worked in a group for longer than a few weeks. In addition, I had never programmed in Python before. Due to these factors, this class presented a very steep learning curve. As the entire class, as well as all the labs, were to build up towards the final project, I will also include my learning experience from the class in general.

From working on all of the labs, homeworks, and the final project, I was able to learn and become proficient in coding in python. In addition, I learned how to use many Python packages, such as NumPy, matplotlib, etc. In addition, I got a lot of experience with executing a lot of commands on the terminal and with working with Docker.

Because we were using ROS2's node system with publishers and subscribers, I was able to learn how to program and solve problems in an event-driven way. This way of problem-solving was vastly different from the sequential programming I was used to before this class. When I first started to code the final project, I was still in the sequential programming mindset; however, I quickly realized that it would be very faulty and would not work properly with the sheer amount of publishers and subscribers, so I had to pivot and refactor the final project code to be mostly event-driven. In addition, while working on the final project, I learned how to make custom message types to encapsulate multiple data values into the message. This was incredibly useful as I used to all my desired data values into a long string message that was very error-prone.

This project was an amazing learning experience about hardware and how code interacts with it. It was the first time I had to code with real hardware, instead of just toy-like circuits, and it really exposed me to how error-prone hardware projects are. It also improved the way I had to debug, as almost every time I worked on the robot, there would be errors.

However, I think the most impactful learning experience from this project was learning to work effectively on a team for a long period of time. As we only had one Raspberry Pi to code on, the project was essentially partner programming, with one person coding and the other advising. There was a lot of compromising when it came to the code and the methods to use to solve the problems.

Peer Rating

Justin Trinh: 10/10 Kevin Fregoso: 10/10
